

Spec-driven development (SDD)

Specifications

(Structured?)

Executable sources of truth

SDD: Github Copilot

Generating directly from the beginning

1. Procedure

Spec. Kit

Project documentation

Specification file(s)

Technical plans(s)

Implementation task(s)

2. Copilot

uses

Generating aligned, production-ready code

Need to check before moving forward!
(accumulated?)

⇒ "four distinct phases": Specify, plan, tasks, and implement

3. Reasons

"like coding"

a chunk of code can be generated fast + but might not be misaligned

⇒ Concept: Agile principles

o Differences:

1. Not mean writing a massive spec and never changing it → starting a clear spec and can be evolved.
2. Treating each user story with a micro-cycle

→ Lacking information of previous decisions / general project requirements

Why "SDD" offers a better approach

1. Concentrating primarily on: "code over specifications"
2. Solving directly "data organizing issue" making specified executable ???

Summary